Raport projektowy
Uczenie się ze wzmocnieniem

Patryk Kubik

2026/2027

Spis treści

1 Informacje podstawowe **2**

2 Temat i cel projektu **2**
2.1 Opis problemu . . . 2
2.2 Cel agenta . . . 2
2.3 Uzasadnienie zastosowania uczenia się ze wzmocnieniem . . . 2

3 Formalizacja problemu RL/MDP **3**
3.1 Agent i środowisko . . . 3
3.2 Stan lub obserwacja . . . 3
3.3 Akcje . . . 3
3.4 Funkcja nagrody . . . 4
3.5 Warunki zakończenia epizodu . . . 4
3.6 Założenia i ograniczenia środowiska . . . 5

4 Implementacja **5**
4.1 Struktura repozytorium . . . 5
4.2 Środowisko . . . 5
4.3 Agent bazowy . . . 5
4.4 Agent uczący się . . . 5

5 Plan eksperymentów **6**

6 Wyniki **6**

7 Analiza i wnioski **6**

8 Instrukcja uruchomienia **7**

9 Wsparcie AI **7**

10 Literatura **7**

1 Informacje podstawowe

Element	Informacja
Imię i nazwisko	Patryk Kubik
Numer albumu	179538
Kierunek/specjalność	Informatyka/Sztuczna Inteligencja
Tytuł projektu	Zastosowanie uczenia ze wzmocnieniem w grze typu Flappy Bird
Link do repozytorium	https://github.com/PATER10/flappy-bird-reinforcement-learning-project
Data oddania	Do uzupełnienia

2 Temat i cel projektu

2.1 Opis problemu

Projekt polega na rozwiązaniu problemu nieskończonego horyzontu (survivalowego) w dynamicznym środowisku przewijanym bocznie (2D). Agent sterujący obiektem (ptakiem) musi poruszać się w przestrzeni, podlegając wirtualnej grawitacji, i unikać kolizji z pojawiającymi się na jego drodze przeszkodami (rurami). Zderzenie z dowolną przeszkodą lub podłożem kończy próbę.

2.2 Cel agenta

Celem agenta jest maksymalizacja skumulowanej nagrody, co przekłada się na przetrwanie jak najdłużej bez kolizji z przeszkodami i zdobycie jak najwyższego wyniku punktowego. Działanie agenta uznaje się za skuteczne, gdy potrafi on w pełni autonomicznie, bez interwencji człowieka, omijać przeszkody i utrzymywać się w wyznaczonych granicach środowiska przez określoną z góry liczbę wygenerowanych rur, zapobiegając szybkiemu zakończeniu epizodu.

2.3 Uzasadnienie zastosowania uczenia się ze wzmocnieniem

Problem gry Flappy Bird stanowi klasyczny przykład procesu decyzyjnego, w którym uczenie się ze wzmocnieniem (RL) ma ogromne zastosowanie, ponieważ:

- agent musi sekwencyjnie i w czasie rzeczywistym podejmować decyzję o wykonaniu skoku lub powstrzymaniu się od niego na podstawie zmieniających się współrzędnych przeszkód,
- każda decyzja (np. skok) bezpośrednio wpływa na przyszły stan środowiska – zbyt wczesny lub zbyt późny skok może doprowadzić do nieuchronnej kolizji w kolejnych klatkach symulacji,
- istotny jest wynik długoterminowy, a nie natychmiastowa nagroda – pojedynczy skok może nie przynieść od razu punktu (nagrody), ale jest niezbędny do uniknięcia uderzenia w rurę oddaloną o kilkadziesiąt pikseli,

- problem ten cechuje się bardzo dużą, ciągłą przestrzenią stanów. Zastosowanie agenta uczącego się (w szczególności metod Deep RL) jest uzasadnione, ponieważ agent jest w stanie sam wywnioskować optymalną politykę sterowania, bez konieczności ręcznego programowania reguł „jeżeli ptak jest x centymetrów od rury, to skacz", które byłyby nieskuteczne.

3 Formalizacja problemu RL/MDP

3.1 Agent i środowisko

Agentem w tym problemie jest model algorytmiczny (oparty na głębokiej sieci neuronowej), który odpowiada za sterowanie ptakiem. Środowiskiem jest fizyczny silnik gry stworzony w Unity, który zarządza grawitacją, ruchem obiektu i generowaniem przeszkód. W jednym kroku epizodu:

1. Środowisko mierzy aktualne współrzędne ptaka i rur, po czym przesyła te dane (stan) do agenta poprzez interfejs API.
2. Agent na podstawie otrzymanych danych liczbowych podejmuje decyzję o akcji.
3. Agent wysyła akcję z powrotem do środowiska.
4. Środowisko (Unity) wykonuje ruch, sprawdza kolizje i wylicza nagrodę, po czym cały proces się powtarza dla nowej klatki animacji.

3.2 Stan lub obserwacja

W tym przypadku agent otrzymuje obserwację, która bardzo dokładnie odwzorowuje rzeczywisty stan środowiska ważny do podjęcia decyzji.

Składowa stanu/obserwacji	Znaczenie
`bird_y`	Współrzędna Y położenia ptaka na ekranie.
`bird_velocity`	Aktualna prędkość opadania lub wznoszenia ptaka.
`pipe_dist_x`	Odległość w poziomie do najbliższej nadciągającej rury.
`pipe_gap_y`	Współrzędna Y środka wolnej przestrzeni (szczeliny) pomiędzy dolną a górną rurą.

3.3 Akcje

Zbiór akcji dostępnych dla agenta jest dyskretny i bardzo mały. W każdym kroku decyzyjnym agent ma do dyspozycji jedną z dwóch akcji. Żadna z nich nie jest zablokowana w zależności od stanu.

Akcja	Opis
`0` (Brak akcji)	Ptak nie podejmuje żadnego działania, opadając swobodnie pod wpływem grawitacji.
`1` (Skok)	Agent uruchamia impuls siły działający na ptaka pionowo w górę, nadając mu natychmiastową prędkość wznoszenia.

3.4 Funkcja nagrody

Funkcja nagrody została zaprojektowana w taki sposób, aby promować przetrwanie i unikanie przeszkód.

$$r_t = \begin{cases} -100 & \text{za kolizję (rura, ziemia, wyjście poza ekran),} \\ +10 & \text{za pomyślny przelot przez szczelinę pomiędzy rurami,} \\ +0.1 & \text{za każdy przeżyty krok (brak kolizji i brak przelotu).} \end{cases} \tag{1}$$

W tej sekcji wyjaśniono:

- **za co agent otrzymuje nagrodę dodatnią:** Agent otrzymuje dużą nagrodę dodatnią (+10) za bezpieczne pokonanie przeszkody (przekroczenie współrzędnej X rury). Otrzymuje również bardzo małą nagrodę (+0.1) ułamkową za samo utrzymanie się przy życiu w każdej klatce symulacji.
- **za co agent otrzymuje karę:** Agent otrzymuje dotkliwą karę (-100) za zderzenie z rurą, uderzenie w podłoże, a także za wylecenie poza górną krawędź ekranu.
- **czy występuje kara za krok:** W klasycznym ujęciu, gdzie celem jest jak najszybsze dotarcie do mety, stosuje się ujemną karę za krok. Ponieważ w grze Flappy Bird celem jest przetrwanie (nieskończony horyzont), zamiast ujemnej kary za krok zastosowano minimalną nagrodę dodatnią za krok, zachęcającą agenta do unikania stanów terminalnych.
- **czy występują ograniczenia twarde lub miękkie:** Ograniczenia twarde stanowią fizyczne kolidery środowiska – obiekty rur oraz podłoże. Ich naruszenie jest niedozwolone i natychmiast kończy epizod porażką.
- **czy nagroda nie zachęca agenta do niepożądanego zachowania:** Funkcja nagrody uwzględnia karę za opuszczenie obszaru gry od góry (wylecenie poza ekran). Zapobiega to niepożądanemu zachowaniu, polegającemu na ciągłym skakaniu, aby przelecieć bezpiecznie wysoko nad rurami, omijając w ten sposób przeszkody.

3.5 Warunki zakończenia epizodu

Epizod (jedna próba przejścia gry) ulega natychmiastowemu zakończeniu po wejściu środowiska w stan terminalny. Następuje to w przypadku spełnienia co najmniej jednego z poniższych warunków:

- **Kolizja z przeszkodą:** Punkt kolizyjny ptaka wejdzie w interakcję z prostokątem kolizyjnym dolnej lub górnej rury.
- **Kolizja z podłożem:** Ptak opadnie na poziom ziemi.
- **Opuszczenie obszaru gry:** Ptak wyleci poza górną krawędź ekranu (zabezpieczenie przed omijaniem rur górą).
- **Osiągnięcie celu (limit rur):** Agent z powodzeniem pokona 100 rur bez kolizji. Jest to sztuczny warunek zakończenia wprowadzony w celu uniknięcia nieskończonych epizodów w przypadku wyuczenia się przez agenta optymalnej polityki sterowania.

3.6 Założenia i ograniczenia środowiska

W ramach projektu przyjęto następujące założenia i uproszczenia modelu środowiska:

- **Reprezentacja stanu:** Zrezygnowano z analizy obrazu (pikseli) na rzecz bezpośredniego przekazywania dokładnych współrzędnych przestrzennych. Agent „nie widzi" gry, lecz analizuje wektor liczb. Znacząco zmniejsza to złożoność obliczeniową i pozwala na wykorzystanie prostszej architektury sieci.
- **Uproszczona fizyka:** Model fizyczny środowiska uwzględnia jedynie stałą siłę grawitacji (działającą wzdłuż osi Y) oraz skokowe impulsy siły (akcja skoku). Pominięto efekty aerodynamiczne, takie jak opór powietrza, rotację obiektu czy zmienny wiatr.
- **Komunikacja przez API:** Ponieważ agent (Python) i środowisko (Unity) komunikują się przez zewnętrzny interfejs, pomija się kwestię mikrosekundowych opóźnień sieciowych (tzw. lagów). Zakłada się, że przy symulacji uruchomionej w przyspieszonym czasie wymiana pakietów stanu i akcji zachodzi w sposób w pełni synchroniczny z klatkami fizyki.

4 Implementacja

4.1 Struktura repozytorium

Należy krótko opisać strukturę repozytorium i najważniejsze pliki projektu.

```
repozytorium/
|-- src/
|-- experiments/
|-- report/
|-- results/
‘-- README.md
```

4.2 Środowisko

Należy opisać sposób implementacji środowiska, w szczególności funkcje inicjalizacji epizodu oraz wykonania kroku środowiska.

4.3 Agent bazowy

Należy opisać agenta losowego, heurystycznego albo inny baseline wykorzystany do porównania z agentem uczącym się.

4.4 Agent uczący się

Należy opisać zastosowany algorytm RL lub DRL, jego parametry oraz sposób implementacji. W przypadku Q-tab należy opisać reprezentację tablicy wartości. W przypadku DRL należy opisać wejścia modelu, wyjścia modelu, architekturę oraz sposób uczenia.

5 Plan eksperymentów

Należy opisać eksperymenty zaplanowane w celu oceny działania agenta.

Eksperyment	Co jest zmieniane?	Co jest mierzone?
E1	Do uzupełnienia	Do uzupełnienia
E2	Do uzupełnienia	Do uzupełnienia
E3	Do uzupełnienia	Do uzupełnienia

W tej sekcji należy wskazać:

- liczbę epizodów uczenia,
- liczbę powtórzeń eksperymentu albo uzasadnienie braku powtórzeń,
- parametry uczenia,
- metryki oceny,
- warianty środowiska, map lub funkcji nagrody,
- sposób porównania z agentem bazowym.

6 Wyniki

W tej sekcji należy przedstawić wyniki eksperymentów. Wyniki powinny być pokazane w postaci tabel, wykresów lub czytelnych zestawień.

Wariant	Średni zwrot	Skuteczność	Długość epizodu
Agent bazowy	Do uzupełnienia	Do uzupełnienia	Do uzupełnienia
Agent RL/DRL	Do uzupełnienia	Do uzupełnienia	Do uzupełnienia



Rysunek 1: Przykładowy wykres procesu uczenia agenta.

7 Analiza i wnioski

Należy zinterpretować wyniki. W tej części należy odpowiedzieć m.in. na pytania:

- Czy agent uczący się działa lepiej niż agent bazowy?
- Jak zmieniało się zachowanie agenta w trakcie uczenia?
- Które elementy środowiska były najtrudniejsze?
- Czy funkcja nagrody prowadziła do oczekiwanego zachowania?
- Czy wystąpiły przypadki niepożądanego zachowania agenta?
- Jakie są najważniejsze ograniczenia rozwiązania?

8 Instrukcja uruchomienia

Należy opisać sposób uruchomienia projektu od czystego repozytorium. Przykładowo:

```
git clone https://github.com/...
cd nazwa-repozytorium
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python src/main.py
```

Należy również wskazać, jak odtworzyć eksperymenty oraz gdzie zapisywane są wyniki.

9 Wsparcie AI

W tej sekcji należy obowiązkowo opisać wykorzystanie narzędzi sztucznej inteligencji. Jeżeli student nie korzystał z takich narzędzi, należy wpisać jednoznaczną informację, że nie korzystano ze wsparcia AI.

Data	Narzędzie/model	Cel użycia	Transkrypt / link / zakres
26/10/2026 - 05/11/2026	Gemini 3.1 PRO	Konsultacja w celu wybrania tematu projektu, pomoc w pisaniu 3 pierwszych rozdziałów raportu	https://share.gemini.google/KI2bDKycjaV / Wszystkie treści zostały dogłębnie przeanalizowane i dostosowane do wymagań, zgodne opisy zostały ujęte w raporcie

Należy zapisać transkrypty rozmów z modelami AI wraz z datą oraz nazwą modelu. Jeżeli rozmowa odbywała się poza narzędziami programistycznymi, należy dodatkowo udostępnić całą rozmowę poprzez link, o ile użyte narzędzie umożliwia takie udostępnienie.

Student powinien także wyjaśnić, które elementy wygenerowane lub zasugerowane przez AI zostały samodzielnie sprawdzone, zmodyfikowane albo odrzucone.

10 Literatura

W tej sekcji należy wskazać wykorzystane źródła: książki, artykuły naukowe, dokumentację bibliotek, materiały wykładowe oraz inne źródła użyte podczas pracy nad projektem.

Literatura

[1] R. S. Sutton, A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.